FINAL POSTGRESQL SQL INTERVIEW BOOK

Theory + Query Practice + Scenario Questions | Data Analyst • Data Engineer • AI/ML Fresher

Rule: Try every question yourself before reading the model answer. Type the queries in pgAdmin and explain the logic aloud.

1. SQL & DATABASE FUNDAMENTALS

What is SQL?

Structured Query Language used to create, read, update, delete and analyze data in relational databases.

What is a database?

An organized collection of data that can be stored, managed and queried.

DBMS vs RDBMS?

DBMS manages data; RDBMS stores related data in tables and supports relationships, keys and constraints. PostgreSQL is an RDBMS.

What is a table?

A collection of rows (records) and columns (attributes).

What is a primary key?

A column or combination of columns that uniquely identifies each row; it cannot contain NULL.

What is a foreign key?

A column referencing a key in another table to maintain referential integrity.

What are constraints?

Rules such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK and DEFAULT.

DELETE vs TRUNCATE vs DROP?

```
DELETE removes rows and supports WHERE; TRUNCATE removes all rows efficiently; DROP removes the table object.
```

What is normalization?

Structuring tables to reduce redundancy and update anomalies; 1NF, 2NF and 3NF are common forms.

What is denormalization?

Intentionally adding redundancy to improve read performance or simplify analytics.

UNION vs UNION ALL?

UNION removes duplicates; UNION ALL keeps them and is generally faster.

DISTINCT?

Removes duplicate result rows.

What is a schema?

A namespace organizing database objects such as tables, views and functions.

2. SELECT, FILTERING & SORTING

Show every column and row from product.

```
SELECT * FROM product;
```

Show selected columns.

```
SELECT product_name, price FROM product;
```

What does * mean?

All columns.

Products costing more than 10000.

```
SELECT * FROM product WHERE price > 10000;
```

Products priced 5000–10000.

```
SELECT * FROM product WHERE price BETWEEN 5000 AND 10000;
```

Names containing 'phone'.

```
SELECT * FROM product WHERE product_name ILIKE '%phone%';
```

Names starting with S.

```
SELECT * FROM product WHERE product_name ILIKE 'S%';
```

Highest price first.

```
SELECT * FROM product ORDER BY price DESC;
```

Return only 5 rows.

```
SELECT * FROM product LIMIT 5;
```

WHERE vs HAVING?

WHERE filters rows before grouping; HAVING filters groups after GROUP BY.

How do you test NULL?

Use IS NULL or IS NOT NULL, never = NULL.

Why avoid SELECT * in production?

It can read unnecessary data, increase transfer cost and make dependencies less explicit.

3. AGGREGATIONS & GROUP BY

Common aggregate functions?

COUNT, SUM, AVG, MIN and MAX.

Count rows.

```
SELECT COUNT(*) FROM product;
```

COUNT(*) vs COUNT(column)?

COUNT(*) counts rows; COUNT(column) ignores NULL values.

Average price by category.

```
SELECT category, AVG(price) AS avg_price FROM product GROUP BY category;
```

Categories with average > 20000.

```
SELECT category, AVG(price) AS avg_price FROM product GROUP BY category HAVING AVG(price) > 20000;
```

Maximum salary per department.

```
SELECT department, MAX(salary) FROM employees GROUP BY department;
```

Salaries shared by multiple employees.

```
SELECT salary, COUNT(*) FROM employees GROUP BY salary HAVING COUNT(*) > 1;
```

4. JOINS

INNER JOIN?

Returns rows matching in both tables.

LEFT JOIN?

Returns every left-table row plus matching right rows; unmatched right columns are NULL.

INNER vs LEFT JOIN?

INNER keeps only matches; LEFT preserves all left-side records.

Customer-order join.

```
SELECT c.customer_id,c.name,o.order_id,o.amount FROM customers c JOIN orders o ON c.customer_id=o.customer_id;
```

Customers with no orders.

```
SELECT c.customer_id,c.name FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id WHERE o.order_id IS NULL;
```

SELF JOIN?

A table joined to itself, e.g. employee-to-manager.

CROSS JOIN?

Cartesian product: every row combined with every row.

FULL OUTER JOIN?

Keeps matches and unmatched rows from both tables.

Why can joins create duplicates?

A one-to-many or many-to-many relationship can multiply rows; always verify key cardinality.

5. SUBQUERIES, CTEs & CASE

What is a subquery?

A query nested inside another query; it can return a value, list or derived table.

Employees above company average.

```
SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);
```

What is a CTE?

A named temporary result using WITH, useful for readable multi-step queries.

CTE example.

```
WITH dept_avg AS (SELECT department,AVG(salary) avg_salary FROM employees GROUP BY department) SELECT * FROM dept_avg;
```

What is CASE?

A conditional expression for derived categories/business rules.

Salary bands.

```
SELECT name,salary,CASE WHEN salary>=80000 THEN 'High' WHEN salary>=50000 THEN 'Medium' ELSE 'Low' END AS band FROM
employees;
```

COALESCE?

Returns the first non-NULL expression.

NULLIF?

Returns NULL when two expressions are equal; useful for safe division.

6. WINDOW FUNCTIONS

What is a window function?

Calculates across related rows without collapsing them into one row per group.

ROW_NUMBER vs RANK vs DENSE_RANK?

ROW_NUMBER gives unique sequence numbers; RANK ties and skips numbers; DENSE_RANK ties without gaps.

Rank salaries within department.

```
SELECT name,department,salary,DENSE_RANK() OVER(PARTITION BY department ORDER BY salary DESC) AS rnk FROM employees;
```

Top 3 employees per department.

```
WITH x AS (SELECT e.*,ROW_NUMBER() OVER(PARTITION BY department ORDER BY salary DESC) rn FROM employees e) SELECT * FROM
x WHERE rn<=3;
```

Running total by date.

```
SELECT sale_date,amount,SUM(amount) OVER(ORDER BY sale_date) AS running_total FROM sales;
```

Previous row/month.

```
SELECT month,sales,LAG(sales) OVER(ORDER BY month) AS previous_sales FROM monthly_sales;
```

Why PARTITION BY?

It creates independent groups for the window calculation while retaining individual rows.

7. 25 INTERVIEW PROBLEMS — MEDIUM TO ADVANCED

1. Second-highest salary.

```
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
```

2. Names containing 'a' exactly twice.

```
SELECT * FROM employees WHERE LOWER(name) LIKE '%a%a%' AND LOWER(name) NOT LIKE '%a%a%a%';
```

3. Duplicate records.

```
SELECT email,COUNT(*) FROM customers GROUP BY email HAVING COUNT(*)>1;
```

4. Running total by date.

```
SELECT sale_date,SUM(amount) daily_sales,SUM(SUM(amount)) OVER(ORDER BY sale_date) running_total FROM sales GROUP BY
sale_date;
```

5. Above department average.

```
SELECT e.* FROM employees e JOIN (SELECT department,AVG(salary) avg_salary FROM employees GROUP BY department) a ON
e.department=a.department WHERE e.salary>a.avg_salary;
```

6. Most frequent value.

```
SELECT value,COUNT(*) frequency FROM table_name GROUP BY value ORDER BY frequency DESC LIMIT 1;
```

7. Last 7 days — PostgreSQL.

```
SELECT * FROM table_name WHERE date_col >= CURRENT_DATE - INTERVAL '7 days';
```

8. Count employees sharing salary.

```
SELECT salary,COUNT(*) FROM employees GROUP BY salary HAVING COUNT(*)>1;
```

9. Top 3 per group.

```
WITH x AS (SELECT t.*,ROW_NUMBER() OVER(PARTITION BY group_col ORDER BY score DESC) rn FROM table_name t) SELECT * FROM x
WHERE rn<=3;
```

10. Products never sold.

```
SELECT p.* FROM products p LEFT JOIN order_items oi ON p.product_id=oi.product_id WHERE oi.product_id IS NULL;
```

11. First purchase in last 6 months.

```
WITH f AS (SELECT customer_id,MIN(order_date) first_date FROM orders GROUP BY customer_id) SELECT * FROM f WHERE
first_date>=CURRENT_DATE-INTERVAL '6 months';
```

12. Pivot rows to columns.

```
SELECT customer_id,SUM(CASE WHEN category='A' THEN amount ELSE 0 END) category_a,SUM(CASE WHEN category='B' THEN amount
ELSE 0 END) category_b FROM sales GROUP BY customer_id;
```

13. Month-over-month %.

```
WITH m AS (SELECT DATE_TRUNC('month',order_date) month,SUM(amount) sales FROM orders GROUP BY 1),x AS (SELECT
*,LAG(sales) OVER(ORDER BY month) prev_sales FROM m) SELECT
month,sales,ROUND(100.0*(sales-prev_sales)/NULLIF(prev_sales,0),2) mom_pct FROM x;
```

14. Median salary in PostgreSQL.

```
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary FROM employees;
```

15. 3+ consecutive login days.

Use a gaps-and-islands method: row-number each user's dates, group by login_date - row_number*INTERVAL '1 day', then keep groups with COUNT(*)>=3.

16. Delete duplicate rows keeping one.

```
WITH d AS (SELECT ctid,ROW_NUMBER() OVER(PARTITION BY email ORDER BY ctid) rn FROM customers) DELETE FROM customers c
USING d WHERE ctid=d.ctid AND rn>1;
```

17. Sales ratio between categories.

```
SELECT SUM(amount) FILTER(WHERE category='A')/NULLIF(SUM(amount) FILTER(WHERE category='B'),0) AS ratio FROM sales;
```

18. Recursive hierarchy.

```
WITH RECURSIVE org AS (SELECT employee_id,name,manager_id,0 level FROM employees WHERE manager_id IS NULL UNION ALL
SELECT e.employee_id,e.name,e.manager_id,o.level+1 FROM employees e JOIN org o ON e.manager_id=o.employee_id) SELECT *
FROM org;
```

19. Find gaps in numbering.

```
SELECT id+1 AS gap_start FROM table_name t WHERE NOT EXISTS(SELECT 1 FROM table_name x WHERE x.id=t.id+1) AND id<(SELECT
MAX(id) FROM table_name);
```

20. Split comma-separated string.

```
SELECT TRIM(value) FROM unnest(string_to_array('SQL,Python,Excel',',')) AS value;
```

21. Rank products by sales for each region.

```
SELECT region,product_id,SUM(amount) sales,RANK() OVER(PARTITION BY region ORDER BY SUM(amount) DESC) rnk FROM sales
GROUP BY region,product_id;
```

22. Top 10% salary in department.

```
SELECT * FROM (SELECT e.*,CUME_DIST() OVER(PARTITION BY department ORDER BY salary DESC) cd FROM employees e)x WHERE
cd<=0.10;
```

23. Orders during 9 AM–6 PM.

```
SELECT * FROM orders WHERE order_time::time BETWEEN TIME '09:00' AND TIME '18:00';
```

24. Users active on weekdays and weekends.

```
SELECT user_id FROM logins GROUP BY user_id HAVING COUNT(*) FILTER(WHERE EXTRACT(ISODOW FROM login_date) BETWEEN 1 AND
5)>0 AND COUNT(*) FILTER(WHERE EXTRACT(ISODOW FROM login_date) IN (6,7))>0;
```

25. Customers buying 3+ categories.

```
SELECT customer_id FROM orders GROUP BY customer_id HAVING COUNT(DISTINCT category)>=3;
```

8. DATA ANALYST CASE-BASED QUESTIONS

How investigate a sales drop?

Check data freshness/completeness, then compare sales by date, product, region, channel and segment; inspect returns/cancellations; identify when/where the decline began; validate against source.

How handle duplicates?

Define the business duplicate key, detect with GROUP BY/HAVING or ROW_NUMBER, then apply a documented business rule before deleting/retaining.

How handle NULLs?

Understand why they are missing, then filter, replace, derive or retain NULL based on business meaning; document the rule.

Dashboard number doesn't match SQL — what do you do?

Check filters, date range, join multiplication, aggregation grain, NULLs, source refresh and definitions; reconcile step by step.

Validate a SQL result?

Check row counts, totals, distinct keys, NULL rates, sample rows, ranges and reconciliation with a trusted source.

Monthly revenue query.

```
SELECT DATE_TRUNC('month',order_date) month,SUM(amount) revenue FROM orders GROUP BY 1 ORDER BY 1;
```

Product % of total sales.

```
SELECT product_id,SUM(amount) sales,ROUND(100.0*SUM(amount)/SUM(SUM(amount)) OVER(),2) pct_total FROM orders GROUP BY
product_id;
```

What makes a good analytics query?

Correct grain and business logic, clear joins, readable aliases, safe NULL/zero handling, and acceptable performance.

9. DATA ENGINEER / PERFORMANCE

What is an index?

A structure that can speed up lookups, joins and some sorts, at the cost of storage and write overhead.

When add an index?

On columns frequently used for selective filters/joins or suitable ordering, after checking actual workload and query plans.

EXPLAIN?

Shows PostgreSQL's planned execution strategy.

EXPLAIN ANALYZE?

Executes the query and reports actual execution details, useful for diagnosing performance.

Slow query approach?

Use EXPLAIN ANALYZE; inspect scans/joins; reduce unnecessary rows/columns; check indexes/statistics; simplify repeated work; benchmark again.

What is a transaction?

A group of operations treated as one logical unit.

What is ACID?

Atomicity, Consistency, Isolation and Durability.

View vs materialized view?

A view stores the query definition; a materialized view stores the query result and can be refreshed.

ETL vs ELT?

ETL transforms before loading; ELT loads first and transforms in the target system.

Why is join cardinality important?

It determines whether a join preserves, reduces or multiplies rows and prevents incorrect aggregates.

10. POSTGRESQL-SPECIFIC

Why did DESCRIBE random; fail?

DESCRIBE is not PostgreSQL SQL syntax. In psql use \d random; in pgAdmin inspect Columns or query information_schema.

Current database.

```
SELECT current_database();
```

List public tables.

```
SELECT table_name FROM information_schema.tables WHERE table_schema='public';
```

List random table columns.

```
SELECT column_name,data_type FROM information_schema.columns WHERE table_schema='public' AND table_name='random' ORDER
BY ordinal_position;
```

Show random data.

```
SELECT * FROM random;
```

VARCHAR vs TEXT?

Both support variable-length strings; TEXT has no declared length limit. Choose based on business validation needs.

ILIKE?

PostgreSQL case-insensitive pattern matching.

FILTER on aggregates?

Adds a condition to an aggregate, e.g. COUNT(*) FILTER (WHERE status='paid').

DATE_TRUNC?

Truncates date/timestamp to a unit such as month, day or year.

INTERVAL?

Represents a time duration, e.g. INTERVAL '7 days'.

11. TRICK / RAPID-FIRE

Can a primary key be NULL?

No.

Can a table have multiple primary keys?

No; one primary-key constraint, though it may contain multiple columns.

Can it have multiple UNIQUE constraints?

Yes.

Does COUNT(column) count NULL?

No.

Can aggregate functions normally be used in WHERE?

No; use HAVING or a subquery/CTE.

What happens with NULL + 10?

NULL, because the result is unknown.

What is a Cartesian product?

All possible row combinations between two sets.

What is aliasing?

Giving a temporary name to a table/column, e.g. employees e.

What is cardinality?

Depending on context, the number of rows, distinct values, or relationship multiplicity.

12. MOCK INTERVIEW — ANSWER OUT LOUD

Tell me about yourself and your SQL experience.

60–90 seconds: engineering background → analytics/data skills → PostgreSQL practice → project → business impact.

Explain WHERE vs HAVING.

WHERE filters rows before grouping; HAVING filters aggregated groups.

Second-highest salary without LIMIT/OFFSET.

Use a subquery or DENSE_RANK and explain how ties are treated.

Customers who never ordered.

LEFT JOIN orders and filter WHERE order_id IS NULL.

Explain ROW_NUMBER/RANK/DENSE_RANK.

State tie behavior and when each is useful.

JOIN creates duplicates — why?

Likely non-unique join keys or one-to-many/many-to-many relationships; check cardinality.

Top 3 products per region?

Use ROW_NUMBER/RANK with PARTITION BY region.

Query takes 30 seconds — what first?

Run EXPLAIN ANALYZE and inspect scans, joins, filters and indexes.

How do you validate analysis?

Validate assumptions, grain, row counts, totals, NULLs, duplicates and source reconciliation.

Explain a SQL project end-to-end.

Source → schema → cleaning → queries → validation → analysis → dashboard/output → insight.

13. FINAL REVISION CHECKLIST

Before interview

```
SELECT/WHERE/ORDER BY/LIMIT • GROUP BY/HAVING • aggregates • INNER/LEFT/FULL/SELF JOIN • subqueries • CTEs • CASE •
NULL/COALESCE • window functions • Top-N • running totals • dates • duplicates • gaps • recursive CTE • PostgreSQL syntax
• EXPLAIN ANALYZE • indexes • business cases • project explanation.
```